

**LAPORAN PRATIKUM
PEMOGRAMAN BERORIENTASI OBJEK**

LAPORAN PRAKTIKUM PEKAN 2

**Disusun Oleh:
Muhammad Aufa Rafiki
2511531012**

**Dosen Pengampu: Nurfiah, S.ST, M.Kom
Asisten Praktikum: Ezza Addini**



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

PEMBAHASAN

A. Langkah Kerja Praktikum

kita akan membuat objek Transaksi setiap kali terjadi aktivitas penyetoran atau penarikan tunai, lalu menyimpannya di dalam objek Rekening.

1. Membuat Class Diagram Transaksi

Langkah Kerja:

- 1) Buat class dengan nama Transaksi

```
public class Transaksi {
```

- 2) Buat atribut dan constructor pada class Transaksi

Class Transaksi memiliki 3 atribut yaitu idTransaksi, jenis dan nominal. Jenis transaksi ada 2 buah yaitu Debit digunakan untuk penarikan dan Kredit digunakan untuk penyetoran.

```
String idTransaksi;  
String jenis;  
double nominal;  
  
//constructor  
public Transaksi(String id, String jenis, double nominal) {  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

Penjelasan Program:

- String idTransaksi; : Mendeklarasikan variabel idTransaksi bertipe String untuk menyimpan kode identitas unik transaksi.
- String jenis; : Mendeklarasikan variabel jenis bertipe String untuk menyimpan jenis atau kategori transaksi
- double nominal; : Mendeklarasikan variabel nominal bertipe double untuk menyimpan nilai uang transaksi.
- Konstruktor
 - Method khusus yang otomatis dijalankan saat Anda membuat objek baru dari kelas Transaksi
 - Meminta 3 nilai parameter awal (id, jenis, nominal) untuk langsung diisikan ke atribut kelas menggunakan kata kunci this
 - this.idTransaksi = id; : Memasukkan nilai dari variabel masukan

id ke dalam atribut idTransaksi milik objek ini.

- this.jenis = jenis; : Memasukkan nilai dari variabel masukan jenis ke dalam atribut jenis milik objek ini.
- this.nominal = nominal; : Memasukkan nilai dari variabel masukan nominal ke dalam atribut nominal milik objek ini.

3) Membuat method

Kita akan membuat method cetakDetail() yang nantinya akan digunakan untuk menampilkan 1 baris detail transaksi.

```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);  
}
```

Penjelasan Program:

- Sebuah method untuk menampilkan seluruh informasi/detail data transaksi ke layar terminal dalam satu baris teks ringkas.
- Mencetak informasi gabungan dari ID, jenis, dan nominal transaksi ke layar terminal dalam satu baris.

Kode Program lengkap Class Transaksi.

```
1 package Praktikum2_2511531012_MuhammadAufaRafiki;  
2  
3 public class Transaksi {  
4     String idTransaksi;  
5     String jenis;  
6     double nominal;  
7  
8     //konstruktor  
9     public Transaksi(String id, String jenis, double nominal) {  
10         this.idTransaksi = id;  
11         this.jenis = jenis;  
12         this.nominal = nominal;  
13     }  
14  
15     public void cetakDetail() {  
16         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);  
17     }  
18  
19 }  
20
```

2. Membuat Relasi pada Rekening.java

Buka kembali file Rekening.java dari Praktikum sebelumnya. Kita akan menambahkan atribut baru yang merepresentasikan relasi has-a yaitu satu rekening memiliki banyak transaksi.

```
import java.util.ArrayList;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    //Implementasi asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening(String nomor, String nama, double saldoAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        //wajib menginisialisasi ArrayList di dalam Konstruktor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
    }
}
```

Penjelasan Program:

- import java.util.ArrayList; : Mengimpor kelas ArrayList untuk membuat daftar transaksi yang ukurannya dapat bertambah secara dinamis.
- Deklarasi atribut → nomorRekening, namaPemilik, saldo : Atribut untuk menyimpan data dasar rekening (nomor, nama pemilik, dan jumlah saldo).
- ArrayList<Transaksi> riwayatTransaksi; : Mendeklarasikan variabel riwayatTransaksi berbentuk ArrayList untuk menampung kumpulan objek Transaksi.
- Method Konstruktor
 public Rekening(String nomor, String nama, double saldoAwal) {}
 - Inisialisasi Data: Mengisi nilai atribut nomorRekening, namaPemilik, dan saldo sesuai parameter yang dimasukkan saat pembuatan objek.
 - Inisialisasi List → this.riwayatTransaksi = new ArrayList<>();
Membuat dan menginisialisasi objek ArrayList baru yang kosong ke dalam atribut riwayatTransaksi.
 - Cetak Status: Menampilkan pesan bahwa rekening baru telah berhasil dibuat beserta nama dan saldo awalnya

3. Mencatat Riwayat Transaksi

Silahkan perbaiki method setorTunai dan tarikTunai di class Rekening.java agar secara otomatis menginstansiasi objek Transaksi dan menyimpannya ke dalam riwayatTransaksi saat transaksi berhasil.

```

public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        //merekam riwayat
        String idTrx = "TRS-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

```

Penjelasan Program:

- method setorTunai dengan parameter nominal bertipe double untuk memproses penambahan saldo.
- if (nominal > 0) { : Memeriksa apakah nominal uang yang disetor bernilai valid (lebih besar dari 0).
 - saldo += nominal; : Menambahkan jumlah nominal penyetoran ke dalam total saldo rekening saat ini.
 - String idTrx = "TRS-S-" + System.currentTimeMillis(); : Membuat kode unik ID transaksi dengan menggabungkan awalan "TRS-S-" dan angka waktu sistem dalam milidetik.
 - Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal); : Menginstansiasi objek Transaksi baru dengan jenis "Kredit" dan memasukkan parameter idTrx serta nominal
 - riwayatTransaksi.add(trxBaru); : Menyimpan objek trxBaru ke dalam daftar riwayatTransaksi (ArrayList).
 - Mencetak pesan konfirmasi ke terminal bahwa proses setor tunai berhasil beserta saldo terbarunya.
- } else { : Menandakan kondisi alternatif jika pemeriksaan awal nominal > 0 tidak terpenuhi (nominal 0 atau negatif).
 - Mencetak pesan peringatan ke terminal jika nominal setor tidak valid.

B. Latihan /Tugas

1. Integrasi Rekam Jejak Penarikan (30%)

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat

jika penarikan berhasil (saldo mencukupi).

Jawaban:

```
public void tarikTunai(double nominal) {
    if (nominal < 10000){
        System.out.println("Gagal: Nominal tarik tunai harus lebih dari 10000!");
    } else if (nominal > saldo) {
        System.out.println( "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit ", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    }
}
```

Penjelasan Program:

- Mendeklarasikan method tarikTunai dengan parameter nominal bertipe double untuk memproses penarikan saldo.
- if (nominal < 10000){ : Memeriksa kondisi pertama apakah nominal penarikan kurang dari batas minimal (10.000).
 - Mencetak pesan gagal jika nominal yang dimasukkan di bawah 10.000.
- } else if (nominal > saldo) { : Memeriksa kondisi kedua apakah nominal penarikan melebihi saldo yang tersedia di rekening.
 - Mencetak pesan gagal beserta sisa saldo saat ini jika saldo tidak mencukupi.
- } else { : Blok yang dieksekusi jika kedua kondisi kegagalan di atas tidak terpenuhi (artinya penarikan berhasil).
 - saldo -= nominal; : Mengurangi nilai saldo dengan nominal penarikan.
 - String idTrx = "TRX-T-" + System.currentTimeMillis(); : Membuat kode ID unik transaksi menggunakan prefiks dan angka waktu sistem dalam milidetik.
 - Transaksi trxBaru = new Transaksi(idTrx, "Debit ", nominal); : Menginstansiasi objek Transaksi baru dengan jenis transaksi debit dan jumlah nominalnya.
 - riwayatTransaksi.add(trxBaru); : Menyimpan objek transaksi baru ke dalam daftar riwayatTransaksi (ArrayList).
 - Mencetak pesan konfirmasi penarikan berhasil beserta jumlah saldo yang tersisa.

2. Fitur Cetak Mutasi Rekening (70%)

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".
- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

Jawaban:

- Method cetakMutasi().

```
public void cetakMutasi() {  
    System.out.println("\n--- Riwayat Transaksi ---");  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening ini");  
    } else {  
        for (Transaksi trx : riwayatTransaksi) {  
            trx.cetakDetail();  
        }  
    }  
}
```

Penjelasan:

- Mendeklarasikan method publik bernama cetakMutasi tanpa nilai kembalian (*void*).
- Mencetak judul header "Riwayat Transaksi" diawali dengan baris baru
- if (riwayatTransaksi.isEmpty()) { : Memeriksa apakah koleksi riwayatTransaksi dalam keadaan kosong (belum ada transaksi).
 - Mencetak pesan pemberitahuan Belum ada transaksi pada rekening ini
- else { : Blok yang dieksekusi jika daftar riwayatTransaksi berisi minimal satu transaksi.
 - for (Transaksi trx : riwayatTransaksi) { : Perulangan *for-each* untuk mengambil setiap objek Transaksi (diberi nama variabel sementara trx) di dalam daftar riwayatTransaksi secara berurutan.
 - trx.cetakDetail(); : Memanggil method cetakDetail() milik objek trx untuk menampilkan informasi detail masing-masing transaksi ke layar.

- Tambahan Menu Baru "6. Cetak Mutasi (Riwayat)"

```
System.out.println("\nMenu Utama:");
System.out.println("1. Buka Rekening Baru");
System.out.println("2. Setor Tunai");
System.out.println("3. Tarik Tunai");
System.out.println("4. Cek Informasi Rekening");
System.out.println("5. Ganti Akun");
System.out.println("6. Cetak Mutasi (Riwayat)");
System.out.println("0. Keluar");
System.out.print("Pilih menu:");
```

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Penjelasan Program:

- System.out.println("6. Cetak Mutasi (Riwayat)"); : Menambahkan pilihan menu ke-6 pada daftar menu utama di konsol terminal.
- Logika Pemrosesan Menu (case 6):
 - case 6: : Pilihan menu yang akan dijalankan jika pengguna memasukkan angka 6.
 - if (akunAktif == null) { : Memeriksa apakah pengguna belum memilih atau membuka rekening aktif.
 - System.out.println("Error: Anda belum membuka rekening!"); : Mencetak pesan kesalahan jika variabel akunAktif masih bernilai null.
 - } else { : Blok yang dieksekusi jika rekening aktif sudah tersedia (akunAktif tidak null).
 - akunAktif.cetakMutasi(); : Memanggil method cetakMutasi() dari objek akunAktif untuk menampilkan seluruh riwayat transaksinya.
 - break; : Menghentikan proses percabangan switch-case agar program tidak berlanjut ke case berikutnya.

Link GitHub:

https://github.com/MuhammadAufaRafiki/PrakPBO_B_2511531012